

Tensor.h & Tensor.c

Complete Beginner's Guide

Every struct, every field, every function — explained with examples and diagrams

Sections in this guide:

- What a tensor is — the concept and the analogy
- Tensor.h — all structs: `shape_t`, `sparsity_t`, `tensor_t`, `parameter_t`
- What a pointer field inside a struct means
- Bit packing — why `BOOL` stores 8 values per byte
- Element counting functions — `calcNumberOfElements...`
- Size calculation functions — `calcBytes/Bits` per element and tensor
- `calcNumberOfBytesForData` — the +7 trick for rounding up
- Bool tensor bit operations — `tensorBoolGet` and `tensorBoolSet`
- Low-level bit manipulation — `getBitmask`, `readByte`, `writeByte`
- `byteConversion` — the bit-width repacking engine
- Getter and setter functions
- `transposeTensor` — what a transpose is and how it works here
- Print functions — `printTensor` and `printShape`
- Copy functions — `copyData`, `copyShape`, `copyQuantization`, `copyTensor`
- Complete worked example and function reference table

1 — What Is a Tensor?

In mathematics and machine learning, a **tensor** is a generalisation of scalars, vectors, and matrices to any number of dimensions.

Name	Dimensions	Example	In code
Scalar	0D	3.14	A single float
Vector	1D	[1.0, 2.0, 3.0]	Array of 3 floats
Matrix	2D	3×4 grid of numbers	Array of 12 floats, shape [3,4]
Tensor	3D+	8×3×4 cube of numbers	Array of 96 floats, shape [8,3,4]

In this project, a tensor is the fundamental data container — weights, biases, activations, and gradients are all stored as tensors. The `tensor_t` struct in this file is how a tensor is represented in C.

★ **Key insight:** In C there are no multi-dimensional arrays with metadata. A `tensor_t` is a flat byte buffer (`uint8_t *data`) plus metadata structs that describe its shape, number format, and sparsity. The 'multi-dimensional' nature is entirely in the shape metadata.

2 — Tensor.h — All Structs Explained

tensorStorageId

```
typedef void *tensorStorageId;
```

A type alias for `void*` — a pointer to anything. Used as an opaque ID to identify where a tensor's data lives in storage. `void*` means the compiler does not care what type is at that address — it is just an address. Cast it to the correct type when you need to use it. Currently used as a placeholder — the actual storage system is `DataStorage`.

shape_t — Describes the Tensor's Dimensions

shape_t

<code>size_t</code>	<code>numberOfDimensions</code>	How many dimensions this tensor has. A vector has 1, a matrix has 2, a 3D tensor has 3.
<code>size_t *</code>	<code>dimensions</code>	Array of size <code>numberOfDimensions</code> . Each entry is the size along that dimension. For a 3×4 matrix: <code>dimensions=[3,4]</code> .
<code>size_t *</code>	<code>orderOfDimensions</code>	Array that maps logical dimension index to physical storage index. Default: <code>[0,1,2,...]</code> . After transpose it may be <code>[1,0,2,...]</code> .

Understanding shape with a concrete example:

```
// A 2D matrix of shape [3, 4]:
numberOfDimensions = 2
dimensions[0] = 3 // 3 rows
dimensions[1] = 4 // 4 columns
orderOfDimensions = [0, 1] // default: row-major
```

```
// Total elements = 3 * 4 = 12
// Total bytes (FLOAT32) = 12 * 4 = 48 bytes
```

What is orderOfDimensions?

In memory, a 3×4 matrix is stored as a flat sequence of 12 values. The question is: do you store all 4 columns of row 0, then row 1, etc. (row-major / C order), or all 3 rows of column 0, then column 1, etc. (column-major / Fortran order)? `orderOfDimensions` encodes this. The default [0,1] means dimension 0 (rows) changes slowest — row-major. After `transposeTensor()`, it becomes [1,0] — column-major. No data moves in memory — only this array changes.

sparsityType_t, sparsityConfig, sparsity_t — Sparse Tensor Support

```
typedef enum {
    SPARSITY_TYPE_1,
    SPARSITY_TYPE_2
} sparsityType_t;
typedef struct sparsityConfig {
} sparsityConfig; // empty!
typedef struct sparsity {
    sparsityType_t type;
    sparsityConfig *config;
} sparsity_t;
```

Sparsity means most values in the tensor are zero. RigL sparse training exploits this — instead of storing all zeros, you only store non-zero values plus a mask of which positions are non-zero.

sparsityConfig is empty — the struct is a placeholder for future sparsity configuration. The framework currently tracks whether sparsity exists (via the `sparsity_t` pointer being non-NULL) but the config details are not yet implemented. If a tensor's sparsity pointer is NULL, the tensor is dense (no sparsity).

tensor_t — The Main Tensor Struct

tensor_t		
<code>uint8_t *</code>	data	Pointer to the raw byte buffer holding all the tensor values. This is the actual data — all other fields are metadata about it.
<code>shape_t *</code>	shape	Pointer to the shape struct — how many dimensions and how big each one is.
<code>quantization_t *</code>	quantization	Pointer to the quantization struct — what number format the data is in (FLOAT32, SYM_INT32, etc.) and the scale/zeroPoint config.
<code>sparsity_t *</code>	sparsity	Pointer to sparsity info, or NULL if the tensor is dense.

Why are all fields pointers?

Every field in `tensor_t` is a pointer (notice the * on every type). This means the tensor struct itself is tiny — just four addresses, 16 bytes total on a 32-bit system. The actual data, shape arrays, and config structs live elsewhere in memory and `tensor_t` just points to them. This design lets multiple tensors share the same shape struct (e.g. a weight tensor and its gradient tensor have the same shape), saving memory.

```
// Memory layout:
tensor_t (16 bytes on STM32):
data → [byte 0][byte 1]...[byte N] ← the actual values
shape → shape_t { dims=[3,4], n=2, order=[0,1] }
quantization → quantization_t { type=SYM_INT32, cfg→{scale=0.00394} }
sparsity → NULL (dense tensor)
```

parameter_t — Weight + Gradient Pair

parameter_t

tensor_t *	param	The weight tensor — the actual learnable parameter values.
tensor_t *	grad	The gradient tensor — stores dLoss/dParam computed during backpropagation. Same shape as param.

In neural network training, every learnable weight has two tensors associated with it: the weight itself and its gradient. The gradient tells the optimiser how to update the weight. `parameter_t` bundles them together so you can pass both around as a single unit.

3 — Element Counting Functions

Before you can allocate memory or loop over values, you need to know how many elements a tensor has. These three functions compute that.

calcNumberOfElementsByShape

```
size_t calcNumberOfElementsByShape(shape_t
*shape) {
    size_t numElem = 1;
    size_t n = shape->numberOfDimensions;
    size_t *dims = shape->dimensions;
    for (size_t i = 0; i < n; i++) {
        numElem *= dims[i];
    }
    return numElem;
}
```

Multiplies all dimension sizes together. Shape [3,4]: $3 \times 4 = 12$ elements. Shape [8,3,4]: $8 \times 3 \times 4 = 96$ elements. **Start at 1** — because multiplying by 1 has no effect. If you started at 0, every result would be 0. This is the fundamental operation — everything else depends on knowing the element count.

```
size_t calcNumberOfElementsByTensor(tensor_t *t)
{
    return calcNumberOfElementsByShape(t->shape);
}

size_t
calcNumberOfElementsByParameter(parameter_t *p)
{
    return
        calcNumberOfElementsByShape(p->param->shape);
}
```

Convenience wrappers — they call `calcNumberOfElementsByShape` with the right pointer. Instead of writing `tensor->shape` every time, you just pass the tensor directly. The parameter version uses `param->shape` because `param` and `grad` always have the same shape.

4 — Size Calculation Functions

Once you know the element count, you need to know how many bytes each element occupies to calculate memory requirements. This depends on the quantization type.

calcBytesPerElement — How many bytes does one value occupy?

```

size_t calcBytesPerElement(quantization_t *q) {
    switch (q->type) {
        case INT32: return sizeof(int32_t); // 4
        case FLOAT32: return sizeof(float); // 4
        case SYM_INT32: return sizeof(int32_t); // 4
        case SYM: {
            symQConfig_t *symQC = q->qConfig;
            return ceilf(symQC->qBits / 8.0f);
        }
        case ASYM: {
            asymQConfig_t *aqc = q->qConfig;
            return ceil(aqc->qBits / 8.0f);
        }
        case BOOL: return 1;
    }
}

```

switch on the type tag — returns how many bytes one element uses. INT32/FLOAT32/SYM_INT32: always 4 bytes. SYM/ASYM: variable — depends on qBits. $\text{ceilf}(q\text{Bits}/8)$ rounds UP to the nearest whole byte: 4 bits $\rightarrow \text{ceil}(0.5) = 1$ byte 8 bits $\rightarrow \text{ceil}(1.0) = 1$ byte 12 bits $\rightarrow \text{ceil}(1.5) = 2$ bytes. BOOL: 1 byte is returned, BUT this is misleading — actually 8 BOOL values share one byte (1 bit each). calcNumberOfBytesForData handles BOOL correctly with $(n+7)/8$.

Type	calcBytesPerElement	calcBitsPerElement
INT32	4	32
FLOAT32	4	32
SYM_INT32	4	32
SYM (8-bit)	1	8
SYM (4-bit)	1	4
ASYM (8-bit)	1	8
BOOL	1 (misleading)	1

calcBitsPerElement — For packed formats

Same structure as calcBytesPerElement but returns bits instead. This is the accurate value for SYM and ASYM — a 4-bit format returns 4, not 1 byte's worth (8 bits). Used by byteConversion to know exactly how many bits per value.

calcNumberOfBytesForData — Total bytes for N elements

```

case FLOAT32:
    return numberOfElements * sizeof(float);
case SYM:
    return (calcBitsPerElement(q) * n + 7) / 8;
case BOOL:
    return (numberOfElements + 7) / 8;

```

The +7 trick for ceiling division: Integer division always rounds DOWN. To round UP: $(n + 7) / 8$ instead of $n / 8$. Example: 9 BOOL values need 9 bits = 2 bytes. $9 / 8 = 1 \leftarrow \text{WRONG (truncated)}$ $(9+7) / 8 = 16/8 = 2 \leftarrow \text{CORRECT}$ General formula for ceiling division: $\text{ceil}(a/b) = (a + b - 1) / b$ With $b=8$: $(a + 7) / 8$

5 — BOOL Tensor Bit Operations

BOOL tensors pack 8 boolean values into one byte — one bit per value. This is used for sparsity masks (which weights are zero, which are non-zero). A mask for 1024 weights needs only $1024/8 = 128$ bytes instead of 1024 bytes.

tensorBoolGet — Read one bit

```
bool tensorBoolGet(tensor_t const *tensor,
size_t flatIndex) {
size_t byteIndex = flatIndex / 8;
size_t bitIndex = flatIndex % 8;
return ((tensor->data[byteIndex]
>> bitIndex) & 1u) != 0;
}
```

flatIndex — the position of the bool value in the logical array. **byteIndex = flatIndex / 8** — which byte contains this bit. **bitIndex = flatIndex % 8** — which bit within that byte (0=rightmost). **>>** **bitIndex** — right-shift so the target bit moves to position 0. **& 1u** — mask off all other bits, leaving only the lowest bit. **!= 0** — convert to bool (true/false).

Concrete example — reading bit at flatIndex=10:

```
flatIndex = 10
byteIndex = 10 / 8 = 1 ← look in byte 1
bitIndex = 10 % 8 = 2 ← look at bit position 2

byte 1 = 0b00101101 = 0x2D
>> 2 = 0b00001011 ← shift right by 2
& 1 = 0b00000001 = 1 ← bit was 1
result = true
```

tensorBoolSet — Write one bit

```
if (value) {
// SET the bit to 1:
data[byteIndex] |= (1u << bitIndex);
} else {
// CLEAR the bit to 0:
data[byteIndex] &= ~(1u << bitIndex);
}
```

1u << bitIndex — creates a bitmask with exactly one bit set. **bitIndex=2** → 0b00000100 **|= (OR assign)** to set a bit: existing: 0b00101001 mask: 0b00000100 result: 0b00101101 ← bit 2 is now 1 **~mask** — inverts all bits of the mask: mask: 0b00000100 **~mask**: 0b11111011 **&= ~mask (AND assign)** to clear a bit: existing: 0b00101101 **~mask**: 0b11111011 result: 0b00101001 ← bit 2 is now 0

6 — Low-Level Bit Manipulation: getBitmask, readByte, writeByte

These three functions are the building blocks for byteConversion. They work at the level of individual bits within a byte.

getBitmask — Create a mask for a range of bits

A bitmask is a number where specific bits are set to 1 and the rest are 0. It is used to isolate specific bits from a byte.

```
uint32_t getBitmask(uint32_t startbit,
uint32_t endbit) {
    // Normalise to within-byte positions
    uint32_t s = startbit - (startbit/8)*8;
    uint32_t e = endbit - (startbit/8)*8;
    uint32_t counter = 0, value = 1;
    for (size_t i = 0; i < 8; i++) {
        if ((i >= s) & (e > i))
            counter += value;
        value *= 2; // value = 2^i
    }
    return counter;
}
```

Purpose: return a byte with 1s in positions [startbit, endbit). Example: getBitmask(2, 5) produces: bit positions: 7 6 5 4 3 2 1 0 mask: 0 0 0 1 1 1 0 0 = 0x1C **value *= 2** — doubles value each loop: 1,2,4,8,16,32,64,128 = 2^0..2^7 **counter += value** — only adds the bit if i is in range [s,e). This effectively builds a number by summing powers of 2 for the selected bit positions.

readByte — Extract bits from a byte

```
uint8_t readByte(uint8_t data,
uint8_t startbit,
uint8_t endbit) {
    uint8_t bitmask = getBitmask(startbit, endbit);
    uint8_t intermediate = data & bitmask;
    intermediate >>= startbit - (startbit/8)*8;
    return intermediate;
}
```

Step 1: Get mask for the bit range. **Step 2:** data & bitmask — zero out all bits outside the range. **Step 3:** Right-shift so the extracted bits start at bit 0. Example: data=0b10110100, startbit=2, endbit=5: mask: 0b00011100 data & mask: 0b00010100 >> 2: 0b00000101 = 5 Returns the numeric value of bits 2-4 of the input byte.

writeByte — Insert bits into a byte

```
uint8_t writeByte(uint8_t existingData,
uint8_t data,
uint8_t startbit,
uint8_t endbit) {
    uint8_t mask = getBitmask(startbit, endbit);
    data <<= startbit; // shift to position
    uint8_t intermediate = data & mask; // mask off overflow
    existingData = intermediate | existingData; // merge
    return existingData;
}
```

Purpose: write specific bits into a byte without disturbing other bits. **data <<= startbit** — shift the data to align it with the target bit positions. **data & mask** — make sure data does not overflow into adjacent bits. **intermediate | existingData** — OR merges the new bits into the existing byte. Example: write value 5 (0b101) into bits 2-4 of byte 0b11000011: 5 << 2 = 0b00010100 & mask = 0b00010100 (mask=0b00011100) | exist= 0b11010111 ← bits 2-4 replaced, others unchanged

7 — byteConversion — The Bit-Width Repacking Engine

This is the most complex function in the file. It converts a packed array of N-bit values to a packed array of M-bit values. For example: convert 32-bit int32 values to 8-bit packed values, or expand 8-bit packed values back to 32-bit.

```
void byteConversion(
    uint8_t *dataIn, // input byte buffer
    size_t dataInBits, // bits per input value
    uint8_t *dataOut, // output byte buffer
    size_t dataOutBits, // bits per output value
    size_t numValues) // how many values to convert
```

dataInBits=32, dataOutBits=8: shrink int32 → uint8. **dataInBits=8, dataOutBits=32:** expand uint8 → int32. **dataInBits=32, dataOutBits=4:** ultra-compact 4-bit packing. This handles any combination — the bit-width can be any number, not just multiples of 8.

How it works — the sliding window approach:

The function maintains a cursor position in both the input and output buffers, tracking the current bit position. It reads bits from the input buffer chunk by chunk and writes them into the output buffer, crossing byte boundaries when needed. The inner while-loop advances both cursors simultaneously, copying at most $\min(\text{bitsLeftInInputElement}, \text{bitsLeftInOutputByte})$ bits per iteration.

```
// Concrete example: 2 values of 12 bits → 2 values of 8 bits
//
// Input: [0xABC] [0x123] (12 bits each, stored in 3 bytes total)
// byte0 byte1 byte2
// 0xBC 0xAA 0x23 (little-endian packing)
//
// byteConversion(input, 12, output, 8, 2)
//
// Output: [0xBC] [0x23] (low 8 bits of each value kept)
// byte0 byte1
//
// The function reads 12-bit chunks, writes 8-bit chunks,
// automatically handling the misaligned boundary.
```

■ **Why does this exist?** When you quantize weights to 4-bit or 8-bit format, the values must be packed tightly — two 4-bit values per byte, or three 4-bit values per 3/2 bytes. `byteConversion` handles all the bit-level bookkeeping so conversion functions can just call `byteConversion(src, 32, dst, 4, n)` and get correctly packed output.

8 — Getter and Setter Functions

These functions read or write fields in structs. They exist to give a clean API — callers use the function name instead of accessing struct fields directly.

Getters — Read a value from a struct

```
tensor_t *getParamFromParameter(parameter_t *p)
{
    return p->param;
}

tensor_t *getGradFromParameter(parameter_t *p) {
    return p->grad;
}
```

Both return a `tensor_t` pointer from a `parameter_t`. These are the simplest possible functions — one line each. They exist so code reads as: `getGradFromParameter(param)` instead of: `param->grad`. The function name makes intent clear and hides the struct internals.

setTensorValues — Wire all four fields at once


```
void setTensorValues(tensor_t *tensor,
uint8_t *data,
shape_t *shape,
quantization_t *quantization,
sparsity_t *sparsity) {
    tensor->data = data;
    tensor->shape = shape;
    tensor->quantization = quantization;
    tensor->sparsity = sparsity;
}
```

Sets all four pointer fields of a `tensor_t` in one call. Without this function you would write four separate assignment lines every time you initialise a tensor. After calling this, the tensor is fully wired — it points to its data buffer, shape, quantization config, and sparsity info.

setTensorValuesForConversion — Quick copy for type conversion

```
void setTensorValuesForConversion(
uint8_t *data,
quantization_t *q,
tensor_t *originalTensor,
tensor_t *outputTensor) {
    outputTensor->data = data;
    outputTensor->shape = originalTensor->shape;
    outputTensor->quantization = q;
    outputTensor->sparsity =
        originalTensor->sparsity;
}
```

Used during type conversion: the output tensor gets a new data buffer and new quantization, but copies shape and sparsity from the input. This avoids manually copying these fields in every conversion function. The shape pointer is SHARED — both tensors point to the same `shape_t`. That is safe because conversion never changes the shape.

setShape, setOrderOfDimsForNewTensor, setParameterValues

```
void setShape(shape_t *shape,
size_t *dims, size_t n, size_t *order) {
    shape->dimensions = dims;
    shape->numberOfDimensions = n;
    shape->orderOfDimensions = order;
}

void setOrderOfDimsForNewTensor(
size_t n, size_t *order) {
    for (size_t i = 0; i < n; i++)
        order[i] = i; // [0, 1, 2, ...]
}

void setParameterValues(parameter_t *p,
tensor_t *param, tensor_t *grad) {
    p->param = param;
    p->grad = grad;
}
```

setShape — wires all three shape fields at once. **setOrderOfDimsForNewTensor** — fills the order array with the default [0,1,2,...]. Call this when creating a new tensor so it starts with standard row-major order. **setParameterValues** — wires param and grad tensors into a `parameter_t`.

9 — transposeTensor — Swapping Dimensions

A transpose swaps two dimensions of a tensor. For a 2D matrix, transposing swaps rows and columns. A 3×4 matrix becomes a 4×3 matrix after transposing dimensions 0 and 1.

```
void transposeTensor(tensor_t *tensor,
size_t dim0Index,
size_t dim1Index) {
if (tensor->shape->numberOfDimensions < 2)
return; // can't transpose a 1D tensor
size_t temp =
tensor->shape->orderOfDimensions[dim0Index];
tensor->shape->orderOfDimensions[dim0Index] =
tensor->shape->orderOfDimensions[dim1Index];
tensor->shape->orderOfDimensions[dim1Index] =
temp;
}
```

Key insight: NO data moves in memory. The raw bytes in `data[]` stay exactly where they are. Only `orderOfDimensions` is changed — two entries are swapped. This is called a **virtual transpose** or **lazy transpose** — the transpose is recorded in the metadata, and the actual reordering happens when the data is read. **temp variable** — classic swap pattern: `temp = a a = b b = temp` Without temp, you would lose one of the values.

What changes in `orderOfDimensions`:

```
// Before transpose of a 3x4 matrix:
orderOfDimensions = [0, 1] // row=dim0, col=dim1

// After transposeTensor(tensor, 0, 1):
orderOfDimensions = [1, 0] // col=dim0, row=dim1

// Now when iteration code reads dim0 it gets the column
// and when it reads dim1 it gets the row -> matrix is transposed
```

10 — Print Functions

printTensor — Display all values with type info

`printTensor` reads the raw byte buffer and prints values in a human-readable format depending on the quantization type. It uses a switch on `q->type` so each format is printed correctly.

```
// For FLOAT32:
float currentElement = readBytesAsFloat(&t->data[i * sizeof(float)]);
printf("%f\n", currentElement);

// For SYM_INT32: also prints the scale
symInt32QConfig_t *symQC = q->qConfig;
printf("scale=%e\n", symQC->scale);
int32_t elem = readBytesAsInt32(&t->data[i * sizeof(int32_t)]);
printf("%i\n", elem);

// For BOOL: uses tensorBoolGet for bit-level reading
printf("%d", tensorBoolGet(t, i) ? 1 : 0);
```

■ Note the `BOOL` case uses `tensorBoolGet()` instead of reading bytes directly — because `BOOL` values are packed 8 per byte and cannot be read with a simple array index.

printShape — Display dimension information

```
void printShape(shape_t *shape) {
    printf("NumberOfDims: %lu\n",
        numberOfDims);
    for (size_t i=0; i<numberOfDims; i++)
        printf("%lu\n", shape->dimensions[i]);
    for (size_t i=0; i<numberOfDims; i++)
        printf("%lu\n",
            shape->orderOfDimensions[i]);
}
```

Prints two arrays: **Dims**: the actual sizes [3, 4].
OrderOfDims: the current traversal order [0,1] or [1,0] after transpose. Useful for debugging when you are not sure what shape a tensor has.

11 — Copy Functions

Four copy functions work together to make a complete deep copy of a tensor. A deep copy means all data is duplicated — the copy is fully independent from the original.

copyData — Copy the raw byte buffer

```
void copyData(tensor_t *dest, tensor_t *src) {
    size_t n =
        calcNumberOfElementsByShape(dest->shape);
    size_t sz = calcNumberOfBytesForData(
        src->quantization, n);
    memcpy(dest->data, src->data, sz);
    if (src->sparsity != NULL) {
        memcpy(dest->sparsity, src->sparsity,
            sizeof(sparsity_t));
    }
}
```

memcpy copies the exact bytes from src to dest. The number of bytes is computed from the quantization type — SYM_INT32 uses 4 bytes/element, BOOL uses 1 bit/element etc.
sparsity NULL check — only copy sparsity if it exists. Copying a NULL pointer would crash. dest->data must already point to an allocated buffer large enough to hold the data.

copyShape — Copy dimension arrays

```
void copyShape(shape_t *dest, shape_t *src) {
    memcpy(dest->dimensions,
        src->dimensions,
        src->numberOfDimensions * sizeof(size_t));
    memcpy(dest->orderOfDimensions,
        src->orderOfDimensions,
        src->numberOfDimensions * sizeof(size_t));
    dest->numberOfDimensions =
        src->numberOfDimensions;
}
```

Copies both arrays (dimensions and orderOfDimensions) element by element. The size to copy is numberOfDimensions * sizeof(size_t) — one size_t per dimension. Also copies numberOfDimensions itself. dest must already have allocated arrays large enough.

copyQuantization — Copy type + config

For types with no config (FLOAT32, BOOL): just copy the type, set qConfig=NULL. For SYM_INT32: copy the type AND deep-copy the symInt32QConfig_t struct using memcpy. **Deep copy of qConfig** means the dest tensor gets its own independent copy of the scale and roundingMode — changing one will not affect the other. Note: only FLOAT32, SYM_INT32, and BOOL are handled — other types hit the default error.

copyTensor — The top-level copy function

Calls all three copy functions in the right order: 1. copyShape first — copyData needs dest->shape to know the element count. 2. copyQuantization — copyData needs src->quantization to know the byte count. 3. copyData last — depends on both shape and quantization. **TODO comment** — sparsity copying is not yet implemented. The dest tensor will have whatever sparsity it was initialised with.

12 — Complete Worked Example

Creating, filling, and using a SYM_INT32 tensor from scratch:

[illegible]

```
// ■■ 4. Wire everything into a tensor_t ████████████████████████████████████  
tensor_t *tensor = reserveMemory(sizeof(tensor_t));  
setTensorValues(tensor, data, shape, quant, NULL);  
  
// ■■ 5. Convert from a float tensor ████████████████████████████████████████  
convertTensor(floatTensor, tensor);  
  
// qCfg->scale is now computed from the data  
  
// ■■ 6. Inspect ████████████████████████████████████████████████████████████  
printTensor(tensor);  
printShape(tensor->shape);  
mcuSimPrintStatus("after tensor alloc");
```

13 — Complete Function Reference

Function	Returns	What it does
calcNumberOfElementsByShape(shape)	size_t	Multiplies all dimension sizes
calcNumberOfElementsByTensor(tensor)	size_t	Calls above with tensor->shape
calcNumberOfElementsByParameter(param)	size_t	Uses param->param->shape
calcBytesPerElement(quant)	size_t	Bytes per value for given qtype
calcBitsPerElement(quant)	size_t	Bits per value for given qtype
calcBytesPerTensor(tensor)	size_t	Total bytes for entire tensor
calcNumberOfBytesForData(q, n)	size_t	Total bytes for n elements of type q
tensorBoolGet(tensor, idx)	bool	Read one bit from BOOL tensor
tensorBoolSet(tensor, idx, val)	void	Write one bit into BOOL tensor
getBitmask(start, end)	uint32_t	Create bit mask for range [start,end)
readByte(data, start, end)	uint8_t	Extract bits from a byte
writeByte(existing, data, s, e)	uint8_t	Insert bits into a byte
byteConversion(in, inBits, out, outBits, n)	void	Repack N values between bit widths
getParamFromParameter(param)	tensor_t*	Returns param->param
getGradFromParameter(param)	tensor_t*	Returns param->grad
transposeTensor(t, d0, d1)	void	Swap two entries in orderOfDimensions
setTensorValues(t, data, shape, q, sp)	void	Wire all four tensor fields
setTensorValuesForConversion(data,q,orig,output)	void	Wire output tensor, copy shape/sparsity
setParameterValues(p, param, grad)	void	Wire param and grad into parameter_t
setOrderOfDimsForNewTensor(n, order)	void	Fill order array with [0,1,2,...]
setShape(shape, dims, n, order)	void	Wire all three shape fields
printTensor(t)	void	Print all values with type info

<code>printShape(shape)</code>	<code>void</code>	Print dimension and order arrays
<code>copyData(dest, src)</code>	<code>void</code>	Deep copy raw bytes + sparsity
<code>copyShape(dest, src)</code>	<code>void</code>	Deep copy dimension arrays
<code>copyQuantization(dest, src)</code>	<code>void</code>	Deep copy type + qConfig struct
<code>copyTensor(dest, src)</code>	<code>void</code>	Full deep copy: shape + quant + data

14 — Cheat Sheet

tensor_t	4-pointer struct: data, shape, quantization, sparsity
shape_t	3-field struct: numberOfDimensions, dimensions[], orderOfDimensions[]
parameter_t	Pairs a weight tensor (param) with its gradient tensor (grad)
sparsity_t	NULL = dense tensor. Non-NULL = sparse (mostly zeros)
orderOfDimensions	Default [0,1,2]. After transpose: entries are swapped. No data moves.
(n+7)/8	Integer ceiling division trick: rounds UP to nearest byte
1u << bitIndex	Creates a mask with exactly one bit set at position bitIndex
data = mask	Sets a bit to 1 (OR operation)
data &= ~mask	Clears a bit to 0 (AND with inverted mask)
data >> n	Right-shift: moves bit n steps toward position 0
byteConversion	Repacks N values from inBits width to outBits width
getBitmask(s,e)	Returns a byte with 1s in bit positions [s,e)
copyTensor	Deep copy: all data duplicated, dest is fully independent
setTensorValues	Wires all four pointer fields of a tensor_t in one call
transposeTensor	Swaps two orderOfDimensions entries — zero data movement
printTensor	Reads raw bytes and prints values formatted by qtype
calcBytesPerTensor	= calcBitsPerElement × numElements / 8